PROJECT

TEXT ENCRYPTION AND DECRYPTION

USING CLASSICAL CRYPTOGRAPHIC ALGORITHMS

Name	Bibash Bhandari

Python • Flask • HTML/CSS

Report Contents

The report is organized to move from the project context and web architecture to the mathematical operation of each cipher, then to implementation considerations, verification, limitations, and proposed extensions.

Section	Coverage	Page
Abstract	Project summary and keywords	3
1. Introduction	Background, objectives, and scope	4
2. Methodology and Implementation	Client-server workflow, Flask, and the HTML/CSS interface	5
3. Algorithm Explanations and Mathematical Logic	Caesar, Vigenère, Rail Fence, and Playfair ciphers	7
4. System Requirements	Hardware and software baseline	9
5. Verification and Testing Plan	Recommended functional checks and test record	10
6. Security Considerations and Limitations	Appropriate use and technical constraints	10
7. Conclusion	Learning outcomes and project significance	11
8. Future Scope	Potential technical extensions	12
References	Technical documentation and cryptography literature	12
Appendix A. Source Code Insertion Guide	Locations for the project's source-code listings	13

Abstract

This project presents a browser-based application for demonstrating text encryption and decryption with four classical cryptographic algorithms: Caesar, Vigenère, Rail Fence, and Playfair. It is implemented in Python, using Flask for server-side request handling and HTML/CSS for the interface. A user enters text, selects an algorithm and operation, and supplies the required numeric key, keyword, or rail count through a web form. The Flask backend receives submitted data, applies the selected cipher routine, and displays the resulting text on the same page. The report explains client-server architecture, each technology's contribution, and mathematical principles behind substitution, polyalphabetic substitution, transposition, and digraph substitution. It outlines hardware and software requirements, proposes verification cases, and identifies limitations. Since classical ciphers are vulnerable to cryptanalytic techniques, the application is carefully designed for learning and demonstration, not for protecting sensitive information. Possible extensions include modern cryptographic algorithms, saved operation history, improved validation, and a testing framework.

Keywords: classical cryptography; Flask; Caesar cipher; Vigenère cipher; Rail Fence cipher; Playfair cipher; web application

1. Introduction

1.1 Background and Motivation

Cryptography provides methods for transforming information so that it can be understood only by an intended recipient or by a party that possesses the appropriate key. In modern systems, cryptographic techniques support confidentiality and can form part of wider mechanisms for integrity, authentication, and secure communication. Encryption alone, however, does not automatically provide all of these security properties.

Classical ciphers predate modern computer security, but they remain valuable teaching tools. Their rules are small enough to follow manually, while their weaknesses make it possible to study key choice, substitution, transposition, and the difference between an algorithm that transforms text and a system that offers meaningful security. Classical techniques are introduced in standard cryptography textbooks as foundational encryption methods (Stallings, 2017).

1.2 Problem Statement

Studying a cipher only through a written formula can make the relationship between an algorithm, its key, and its output difficult to observe. The project addresses this learning need by combining four classical ciphers in a single browser-based interface. The user can submit text, choose an algorithm, provide its associated key material, select encryption or decryption, and view the resulting text on the same page.

1.3 Aim and Objectives

The primary aim is to demonstrate how classical cryptographic transformations can be implemented and presented through a small web application. The objectives are to:

- implement or integrate Caesar, Vigenère, Rail Fence, and Playfair cipher routines in Python;
- provide an HTML form for text, algorithm selection, operation mode, and cipher-specific parameters;
- use Flask to receive form submissions, invoke the selected routine, and return the result to the interface;
- explain the mathematical logic and key requirements of each algorithm; and
- identify the application's educational value, testing needs, and limitations as a security tool.

1.4 Scope

The stated scope is text-based encryption and decryption through a local or browser-accessible web application. The application demonstrates algorithm selection, key entry, request handling, and result presentation. Features such as user accounts, persistent storage, deployment hardening, and audit logging are not specified in the project description and are therefore treated as possible extensions rather than existing capabilities.

2. Methodology and Implementation

2.1 Development Approach

The project follows a client-server design. HTML defines the form controls, CSS controls their presentation, and Flask provides the Python-based request handler. The cipher routines form the application logic: once the submitted algorithm and operation are known, the backend applies the corresponding transformation and prepares the result for display. Keeping the form, request handling, and cipher logic conceptually separate makes the application easier to understand and maintain.

2.2 Request and Response Workflow

1. The browser requests the application page and receives the HTML interface.
2. The user enters a text string, selects one of the four ciphers, chooses Encrypt or Decrypt, and supplies the associated key or parameter.
3. The browser sends the form values to the Flask application using an HTTP POST request.
4. The Flask route reads the submitted fields and dispatches the request to the selected cipher routine.
5. The routine returns transformed text; the application renders the page again with the result visible to the user.

This workflow is consistent with Flask's request model: submitted form values can be read from the request object, and a template can be rendered with values supplied by the route (Pallets Projects, n.d.). The exact names of the project's form fields and functions should match the submitted source code.

2.3 Frontend: HTML and CSS

The HTML template provides the text-entry field, algorithm selector, operation selector, and inputs for the relevant parameters. The supplied interface screenshot shows fields for a Caesar shift, a Vigenère keyword, a Rail Fence rail count, and a Playfair keyword, followed by a control that submits the text for processing. CSS is responsible for the visual presentation, including spacing, sizing, input appearance, and the result area. These presentation choices improve usability, but they do not alter the cryptographic strength of the algorithms.

2.4 Backend: Python and Flask

Python contains the cipher logic, while Flask connects that logic to HTTP requests. A route handles page access and submitted form data. On a POST request, the handler reads the text, algorithm, mode, and key fields; selects the appropriate cipher branch; and passes the computed output into the page template. The same-page result presentation makes the interaction direct and avoids requiring a separate result page.

The provided screenshots show an application file named app.py and an HTML template under a templates directory. The route screenshot also shows request-form access and algorithm-based branching. The full source code was not provided as a separate text file, so the listing locations in Appendix A are intended for the final, complete code rather than a reconstruction from screenshots.

2.5 User Interface Evidence

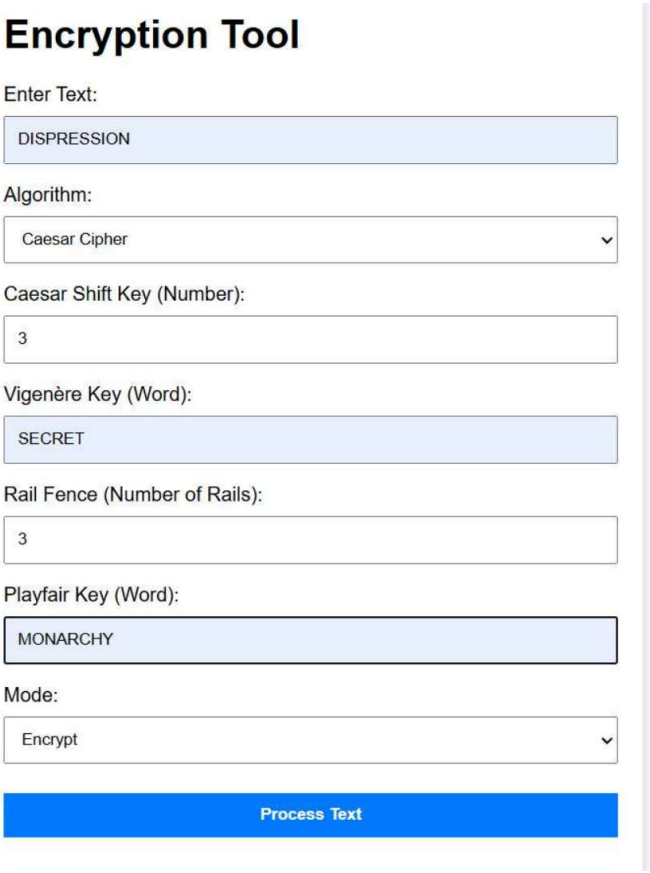
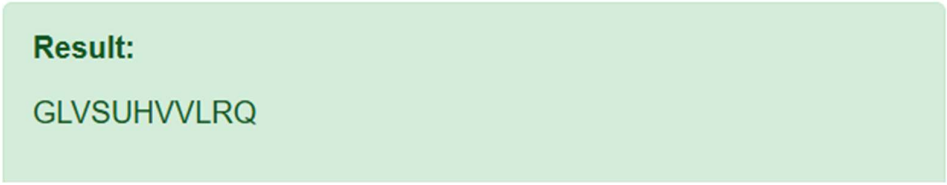


Figure 1. Input form showing the text field, algorithm selector, cipher-specific key fields, operation mode, and submit control.



3. Algorithm Explanations and Mathematical Logic

3.1 Notation

For substitution-cipher equations, each alphabetic letter is represented by an integer from 0 to 25, with A = 0, B = 1, …, Z = 25. Addition and subtraction are performed modulo 26 so that shifts wrap around the alphabet. Let P represent a plaintext letter, C a ciphertext letter, K a key value, and i a position in the message. The equations below describe the conventional forms of the algorithms; handling of letter case, spaces, punctuation, and non-English characters depends on the actual implementation.

3.2 Caesar Cipher

The Caesar cipher is a monoalphabetic substitution cipher. Each alphabetic character is shifted by the same user-defined integer k. Encryption adds the shift; decryption subtracts it. The modulo operation ensures that a shift beyond Z wraps back to A.

$$\text{Encryption: } C = (P + k) \bmod 26$$

$$\text{Decryption: } P = (C - k) \bmod 26$$

For example, with k = 3, HELLO becomes KHOOR: H maps to K, E to H, L to O, and O to R. Applying the inverse shift of 3 to KHOOR restores HELLO. Because there are only 26 possible shifts (including the identity shift), the method is easy to brute-force and should not be considered secure for real messages.

3.3 Vigenère Cipher

The Vigenère cipher is a polyalphabetic substitution method. Instead of applying one shift to every letter, it derives a sequence of shifts from a keyword. The keyword is commonly repeated until it aligns with the plaintext letters; each letter of the key is converted to a value from 0 to 25. This changes the substitution from position to position.

$$\text{Encryption: } C_i = (P_i + K_i) \bmod 26$$

$$\text{Decryption: } P_i = (C_i - K_i) \bmod 26$$

As a simple illustration, HELLO encrypted with the aligned key KEYKE produces RIJVS under the conventional letter-number mapping. Decryption subtracts the corresponding key values and recovers HELLO. Correct alignment of the key is essential; whether spaces and punctuation advance the key position is an implementation-specific rule that should be recorded from the source code.

3.4 Rail Fence Cipher

The Rail Fence cipher is a transposition cipher. It keeps the original symbols but changes their positions. During encryption, the message is written diagonally down and up across a chosen number of rails, forming a zig-zag pattern. The ciphertext is formed by reading the rails row by row. The key is the number of rails rather than a substitution alphabet.

For R rails (R > 1), the zig-zag row pattern repeats with period $2R - 2$. If the character position is i, a conventional row index can be written as $r_i = \min(t, 2R - 2 - t)$, where $t = i \bmod (2R - 2)$. Encryption

groups characters according to these row indices. Decryption reconstructs the same pattern, assigns the ciphertext symbols to the corresponding rail positions, and then reads the grid along the original zig-zag path. A one-rail input is a special case because no rearrangement occurs.

3.5 Playfair Cipher

The Playfair cipher is a digraph substitution cipher: it processes pairs of letters rather than treating every letter independently. A keyword is used to construct a 5 × 5 matrix. Repeated keyword letters are removed, and the remaining alphabet is added to fill the grid. Since a 5 × 5 matrix contains 25 cells, a conventional Playfair square combines I and J into one cell.

Before transformation, plaintext is divided into letter pairs. Traditional Playfair procedures use a filler letter when a pair contains repeated letters and when the message has an odd number of letters. Exact preprocessing conventions vary, so the project's implementation should be checked for its treatment of I/J, duplicate letters, final odd letters, spaces, punctuation, and filler removal.

Let each letter occupy row r and column c in the matrix. For encryption, a pair in the same row is replaced by the letters immediately to its right, wrapping around the row; a pair in the same column is replaced by the letters immediately below, wrapping around the column. For letters forming opposite corners of a rectangle, each letter is replaced by the letter in its own row and the other letter's column. Decryption reverses the row and column directions, while the rectangle rule is applied in the corresponding inverse manner.

3.6 Comparison of the Four Ciphers

Cipher	Cipher type	Key material	Main transformation
Caesar	Monoalphabetic substitution	Integer shift	Moves each letter by a constant offset.
Vigenère	Polyalphabetic substitution	Keyword	Uses changing alphabet shifts based on aligned key letters.
Rail Fence	Transposition	Number of rails	Reorders positions by a zig-zag writing pattern.
Playfair	Digraph substitution	Keyword-derived 5 × 5 square	Transforms two-letter groups using row, column, and rectangle rules.

4. System Requirements

The following table provides a practical baseline for running and demonstrating the application locally. These are recommended planning values, not a claim that the project was benchmarked on a specific machine. The Python version should be selected to match the installed Flask release and the

Category	Requirement	Purpose / Notes
Hardware	Dual-core processor or better	Sufficient for a small local Flask application and basic text transformations.
Hardware	4 GB RAM or more	Adequate for a lightweight development environment; larger capacity improves multitasking.
Hardware	At least 100 MB of free disk space, plus Python environment space	Stores the project files, dependencies, and screenshots; actual needs vary by environment.
Hardware	Display and keyboard	Allows the user to enter text, select options, and inspect results.
Software	Python 3.x compatible with the chosen Flask version	Runs the application logic and web server.
Software	Flask package and its dependencies	Provides routing, request handling, and template rendering.
Software	Modern browser such as Firefox, Chromium, or Brave	Displays the HTML/CSS interface and submits forms.
Software	Code editor or IDE	Used to maintain app.py, templates, stylesheets, and cipher functions.

4.1 Project Components

A typical project layout for the structure shown in the supplied screenshots is:

```
crypto-project/
    app.py
    templates/
        index.html
    static/                 (if CSS is stored separately)
        style.css            (optional)
    requirements.txt        (recommended)
```

The supplied PDF visibly shows app.py and templates/index.html. The static stylesheet and requirements file are shown as a recommended organizational option; their presence in the submitted project should be confirmed before retaining those lines as actual files.

5. Verification and Testing Plan

A report should distinguish design expectations from verified outcomes. The following cases are recommended for manual or automated testing. The Status column is intentionally left open so that each case can be marked only after execution on the final application build.

ID	Test scenario	Expected result	Status
T01	Caesar encryption: HELLO, shift 3	KHOOR under conventional A–Z processing.	[Record]
T02	Caesar decryption: KHOOR, shift 3	HELLO under conventional A–Z processing.	[Record]
T03	Vigenère encryption: HELLO, key KEY	RIJVS using aligned key KEYKE.	[Record]
T04	Vigenère decryption: RIJVS, key KEY	HELLO after reversing the same key alignment.	[Record]
T05	Rail Fence round trip with three rails	Decrypting the ciphertext restores the original input, subject to normalization.	[Record]
T06	Playfair encryption/decryption with a known keyword	Output follows the implemented pair preparation and matrix rules; filler handling is documented.	[Record]
T07	Missing or malformed key	The application should provide a clear response rather than fail unexpectedly.	[Record]
T08	Empty input and boundary cases	The application handles the case predictably and consistently.	[Record]
T09	Spaces, punctuation, and letter case	Observed behavior is documented and matches the intended policy.	[Record]

5.1 Correctness Properties

For reversible cipher functions and valid key material, a central correctness property is that decrypting an encrypted message should reproduce the original message: D(E(P, K), K) = P. For Playfair, the equality may apply to a normalized representation rather than the exact original string because standard preprocessing can merge I/J, insert filler letters, or remove spaces. The implementation's normalization rules should therefore be captured alongside test results.

Before submission, the test table should be completed with actual outputs and a pass/fail status. If a result differs from the expected value, the report should record the observed behavior and the correction made, rather than presenting the expected result as though it had been observed.

6. Security Considerations and Limitations

The application is best understood as an educational demonstration. Caesar encryption has a very small key space and is vulnerable to exhaustive search. Vigenère is more complex than a fixed shift, but classical analysis can exploit repeating key patterns and letter-frequency information. Rail Fence only reorders symbols and does not change their identities. Playfair operates on pairs of letters, but remains a classical cipher with well-known analytical weaknesses. None of these methods should be used to protect passwords, personal information, financial data, or confidential communications.

The project also demonstrates the distinction between application security and cipher design. A working form and a successful POST request do not establish that a cryptographic system is secure. If the web application is exposed beyond a local development environment, additional engineering work would be required, including input validation, production-grade deployment configuration, HTTPS, appropriate request protections, logging policy, and careful handling of keys and submitted text. These improvements would not, by themselves, remove the cryptographic weaknesses of classical algorithms.

The source code was not supplied as a standalone file, and the provided screenshots do not establish the results of every algorithm or edge case. Accordingly, algorithmic correctness, error handling, input normalization, and test coverage should be verified against the complete source and the running application before final submission.

7. Conclusion

The project brings foundational cryptographic ideas into a small, interactive web application. The four included ciphers demonstrate different transformation strategies: Caesar applies a fixed shift, Vigenère varies shifts according to a keyword, Rail Fence rearranges character positions, and Playfair substitutes digraphs using a keyword-derived matrix. Implementing these techniques behind a common interface illustrates how algorithm logic can be connected to user input and output through a web framework.

From a software engineering perspective, the project connects frontend form design, HTTP request handling, backend dispatch, reusable Python functions, and template-based result presentation. From a cybersecurity perspective, it highlights why the mathematical operation of encryption must be evaluated separately from the security of the overall method. The application's strongest educational value lies in making transformations visible and testable while showing why historical ciphers are useful for learning but unsuitable for protecting modern sensitive data.

8. Future Scope

Several extensions could improve the application's reliability, usefulness, and relevance to contemporary cybersecurity practice:

8.1 Modern Authenticated Encryption

Add a modern option such as AES-GCM through a reputable cryptographic library. The interface should clearly distinguish educational classical ciphers from modern authenticated encryption. Key generation, nonce uniqueness, storage, and error handling must follow the library's documented requirements; a cipher name alone is not enough to ensure safe use.

8.2 History and Data Persistence

A lightweight database such as SQLite could store selected operation metadata or, where appropriate, results. Any saved plaintext, ciphertext, or key material would create privacy and security concerns. History should therefore be optional, minimized, and designed so that secrets are not stored unnecessarily.

8.3 Validation and User Experience

The application could validate numeric shifts and rail counts, reject missing keywords, provide informative messages for invalid submissions, and clarify how spaces, punctuation, and letter case are treated. The form could also show only the key field relevant to the selected cipher and provide an accessible, responsive layout for smaller screens.

8.4 Automated Tests and Documentation

Unit tests should cover known input-output pairs, encryption/decryption round trips, invalid keys, empty strings, and boundary conditions. The project could also include a README, a dependency list, setup instructions, and a clear description of the limitations of each algorithm. Reproducible test cases would make future changes safer and easier to evaluate.

8.5 Deployment and Observability

If the application is later deployed for multiple users, it should use a production-appropriate server configuration, HTTPS, request-size limits, access controls where needed, and a privacy-conscious logging policy. Performance measurements and usability feedback could then inform further improvements without confusing successful operation with cryptographic strength.

References

Pallets Projects. (n.d.). Quickstart — Flask Documentation (3.1.x). https://flask.palletsprojects.com/en/stable/quickstart/

Python Software Foundation. (n.d.). Python documentation. https://docs.python.org/3/

Stallings, W. (2017). Cryptography and Network Security: Principles and Practice (7th ed.). Pearson.

Appendix A. Source Code Insertion Guide

Paste the corresponding source code into the boxes below. Use the actual project files and preserve Python indentation. Keep each listing focused on one responsibility; if a listing becomes long, allow it to continue onto the next page or place the complete source code in a separate appendix. The explanatory labels may remain, but remove the “[Paste code here]” prompts before submission.

Listing A.1. Flask application route and request dispatch

Source: app.py. Include imports, Flask app setup, the route handling GET/POST requests, request.form extraction, algorithm/mode dispatch, and template rendering.

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route('/', methods=['GET', 'POST'])
def home():
    result = None
    if request.method == 'POST':
        text = request.form.get('text', '')
        mode = request.form.get('mode')
        algorithm = request.form.get('algorithm')

        if algorithm == 'caesar':
            try:
                shift = int(request.form.get('shift', 3))
                result = caesar_cipher(text, shift, mode)
            except ValueError:
                result = "Error: Please enter a valid number for the shift key."

        elif algorithm == 'vigenere':
            key = request.form.get('vigenere_key', '')
            result = vigenere_cipher(text, key, mode)

        elif algorithm == 'railfence':
            try:
                rails = int(request.form.get('rails', 3))
                result = rail_fence_cipher(text, rails, mode)
            except ValueError:
                result = "Error: Please enter a valid number for the rails."

        elif algorithm == 'playfair':
            key = request.form.get('playfair_key', '')
            result = playfair_cipher(text, key, mode)
```

```
    return render_template('index.html', result=result)

if __name__ == '__main__':
    app.run(debug=True)
```

Listing A.2. Caesar and Vigenère functions

Source: app.py or the relevant cipher module. Include the encryption and decryption functions, key normalization, and any helper used to handle alphabetic characters.

```
def caesar_cipher(text, shift, mode):
    result = ""
    if mode == 'decrypt':
        shift = -shift
    for char in text:
        if char.isalpha():
            start = ord('A') if char.isupper() else ord('a')
            shifted_char = chr((ord(char) - start + shift) % 26 + start)
            result += shifted_char
        else:
            result += char
    return result

def vigenere_cipher(text, key, mode):
    result = ""
    key = key.upper()
    key_index = 0
    key = "".join(filter(str.isalpha, key))
    if not key:
        return "Error: Key must contain at least one letter."
    for char in text:
        if char.isalpha():
            direction = 1 if mode == 'encrypt' else -1
            shift = ord(key[key_index % len(key)]) - ord('A')
            start = ord('A') if char.isupper() else ord('a')
            new_char = chr((ord(char) - start + (shift * direction)) % 26 + start)
            result += new_char
            key_index += 1
        else:
            result += char
    return result
```

Listing A.3. Rail Fence and Playfair functions

Source: app.py or the relevant cipher module. Include the zig-zag rail construction/reconstruction and Playfair matrix generation, pair preparation, encryption, and decryption logic.

```
def rail_fence_cipher(text, rails, mode):
    if rails <= 1:
        return text
    if mode == 'encrypt':
        fence = [""] * rails
        rail = 0
        direction = 1
        for char in text:
            fence[rail] += char
            rail += direction
            if rail == 0 or rail == rails - 1:
                direction *= -1
        return "".join(fence)
    elif mode == 'decrypt':
        pattern = []
        rail = 0
        direction = 1
        for i in range(len(text)):
            pattern.append(rail)
            rail += direction
            if rail == 0 or rail == rails - 1:
                direction *= -1
        rail_counts = [0] * rails
        for r in pattern:
            rail_counts[r] += 1
        fence = []
        index = 0
        for count in rail_counts:
            fence.append(list(text[index:index+count]))
            index += count
        result = ""
        rail_index = [0] * rails
        for r in pattern:
            result += fence[r][rail_index[r]]
            rail_index[r] += 1
        return result

def playfair_cipher(text, key, mode):
```

```
    text = "".join(filter(str.isalpha, text.upper()))
    key = "".join(filter(str.isalpha, key.upper()))
    if not key:
        return "Error: Key must contain letters."
    text = text.replace("J", "I")
    key = key.replace("J", "I")
    alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ"
    grid_chars = []
    for char in key + alphabet:
        if char not in grid_chars:
            grid_chars.append(char)
    grid = [grid_chars[i:i+5] for i in range(0, 25, 5)]

    if mode == 'encrypt':
        pairs = []
        i = 0
        while i < len(text):
            a = text[i]
            b = text[i+1] if i+1 < len(text) else 'X'
            if a == b:
                pairs.append(a + 'X')
                i += 1
            else:
                pairs.append(a + b)
                i += 2
    else:
        pairs = [text[i:i+2] for i in range(0, len(text), 2)]

    def get_coords(char):
        for r in range(5):
            for c in range(5):
                if grid[r][c] == char:
                    return r, c
        return 0, 0

    result = ""
    for pair in pairs:
        r1, c1 = get_coords(pair[0])
        r2, c2 = get_coords(pair[1])
        direction = 1 if mode == 'encrypt' else -1
        if r1 == r2:
```

```
            result += grid[r1][(c1 + direction) % 5]
            result += grid[r2][(c2 + direction) % 5]
        elif c1 == c2:
            result += grid[(r1 + direction) % 5][c1]
            result += grid[(r2 + direction) % 5][c2]
        else:
            result += grid[r1][c2]
            result += grid[r2][c1]
    return result
```

Listing A.4. HTML template

Source: templates/index.html. Include the form fields, algorithm and operation selectors, key inputs, and the template expression used to display the result and any validation messages.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Text Encryption App</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px; background-color: #f4f4f9; }
        .container { background: white; padding: 20px; border-radius: 8px; max-width: 500px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); }
        input, select, button { width: 100%; padding: 10px; margin-top: 8px; margin-bottom: 16px; box-sizing: border-box; }
        button { background-color: #007bff; color: white; border: none; cursor: pointer; font-weight: bold; }
        button:hover { background-color: #0056b3; }
        .result-box { background: #d4edda; color: #155724; padding: 15px; border-radius: 5px; margin-top: 20px; border: 1px solid #c3e6cb; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Encryption Tool</h1>

        <form method="POST">
            <label>Enter Text:</label>
            <input type="text" name="text" required>

            <label>Algorithm:</label>
```

```
        <select name="algorithm">
            <option value="caesar">Caesar Cipher</option>
            <option value="vigenere">Vigenère Cipher</option>
            <option value="railfence">Rail Fence Cipher</option>
            <option value="playfair">Playfair Cipher</option>
        </select>

        <label>Caesar Shift Key (Number):</label>
        <input type="number" name="shift" value="3" min="1" max="25">

        <label>Vigenère Key (Word):</label>
        <input type="text" name="vigenere_key" placeholder="e.g. SECRET">

        <label>Rail Fence (Number of Rails):</label>
        <input type="number" name="rails" value="3" min="2">

        <label>Playfair Key (Word):</label>
        <input type="text" name="playfair_key" placeholder="e.g. MONARCHY">

        <label>Mode:</label>
        <select name="mode">
            <option value="encrypt">Encrypt</option>
            <option value="decrypt">Decrypt</option>
        </select>

        <button type="submit">Process Text</button>
    </form>

    {% if result %}
        <div class="result-box">
            <strong>Result:</strong>
            <p>{{ result }}</p>
        </div>
    {% endif %}
  </div>
</body>
</html>
```

Listing A.5. CSS stylesheet

Source: *The CSS styles are embedded directly within the* <head> *section of the* templates/index.html *file (Listing A.4). They control the form layout, inputs, buttons, and the result display area.*